Tutorial 1 | Classificação supervisionada

Regressão logística e árvore de decisão · Explicação técnica para o docente

1. Problema de pesquisa e desenho analítico

O primeiro tutorial introduz classificação supervisionada: a partir de registros para os quais já se conhece o resultado, um algoritmo aprende uma função que relaciona as características observadas a uma classe de interesse. O caso didático usa 400 registros sintéticos de nascimentos, duas entradas (idade gestacional e número de consultas pré-natais) e uma saída binária, denominada evento (0 = ausência; 1 = ocorrência). O nome do evento remete à mortalidade neonatal apenas para contextualizar a discussão; os dados e a regra de geração não são epidemiologicamente calibrados.

O desenho é preditivo, não causal: pergunta-se se duas características ajudam a distinguir registros com e sem evento. Não se estima o efeito causal do pré-natal ou da duração da gestação. Os registros são simulados por um gerador aleatório com semente fixa; esse procedimento facilita a reprodução em sala, mas não produz uma população representativa.

A regra de simulação

No código, cada pessoa recebe uma idade gestacional inteira de 28 a 41 semanas e entre zero e dez consultas. Uma pontuação linear, definida artificialmente por 1,7 - 0,12 × idade gestacional - 0,08 × consultas, é transformada por uma função logística em probabilidade entre zero e um. O evento é sorteado de uma distribuição de Bernoulli com essa probabilidade. Assim, a regra introduz uma relação esperada entre variáveis e desfecho; o algoritmo não recebe essa regra, apenas observações geradas por ela.

```
p = 1 / (1 + exp(-z))
z = 1.7 - 0.12 * idade_gestacional - 0.08 * consultas_prenatais
```

A escala escolhida no notebook torna o evento relativamente raro. Logo, pode haver poucos casos positivos no treino e no teste, o que faz com que sensibilidade e precisão variem bastante entre diferentes amostras. Não há promessa de alto desempenho: a própria instabilidade permite discutir classes desbalanceadas.

2. Bibliotecas e funções: o papel de cada ferramenta

- NumPy (numpy): fornece vetores numéricos e operações matemáticas vetorizadas. O objeto default_rng gera valores pseudoaleatórios reproduzíveis; exp implementa a exponencial usada na transformação logística; binomial simula a classe final.
- pandas: organiza os vetores em DataFrame, isto é, uma tabela em memória com colunas nomeadas. head() permite inspecionar as primeiras linhas; a seleção X = dados[[...]] e y = dados["evento"] separa entradas e resposta.
- scikit-learn / model_selection: train_test_split cria amostras de treinamento e teste. O argumento test_size=0.30 reserva 30%; random_state fixa o sorteio; stratify=y preserva aproximadamente a

proporção de classes.

- scikit-learn / linear_model: LogisticRegression ajusta os coeficientes que relacionam entradas ao logaritmo das chances do evento. fit() estima os parâmetros; predict() classifica usando o limiar padrão de probabilidade.
- scikit-learn / tree: DecisionTreeClassifier constrói sucessivas divisões no espaço das variáveis; max_depth=3 limita a profundidade. plot_tree produz uma visualização das regras aprendidas.
- scikit-learn / metrics: confusion_matrix contabiliza verdadeiros negativos, falsos positivos, falsos negativos e verdadeiros positivos; recall_score calcula sensibilidade; precision_score calcula precisão.
- Matplotlib: plt.figure, plt.title e plt.show controlam a figura que contém a árvore. Não é uma biblioteca de treinamento; é a camada de visualização.

3. Regressão logística: o que o algoritmo estima

A regressão logística modela a probabilidade da classe 1. Para uma observação com características x1 e x2, constrói uma combinação linear de parâmetros e aplica a função logística. Os coeficientes podem ser relacionados a razões de chances (odds ratios), desde que sua interpretação considere a especificação do modelo, os ajustes realizados e seus pressupostos.

```
log(p / (1 - p)) = beta0 + beta1*x1 + beta2*x2
p = 1 / (1 + exp(-(beta0 + beta1*x1 + beta2*x2)))
```

No notebook, LogisticRegression usa regularização L2 por padrão. Em geral, trata-se de um modelo útil como linha de base, mas com poucos eventos a estimativa pode ficar instável. A predição de classe por predict() usa o limiar padrão; a probabilidade pode ser obtida em aplicações futuras com predict_proba(). A existência de uma associação condicional não prova causalidade.

4. Árvore de decisão: regras e partições

A árvore aprende perguntas do tipo “idade gestacional é menor ou igual a um determinado valor?”. Cada pergunta divide um conjunto de registros em subconjuntos mais homogêneos segundo um critério, como a impureza de Gini. Repetindo essas divisões, a árvore cria ramos e folhas com previsões de classe.

A vantagem didática é a visualização: plot_tree expõe os limiares, os tamanhos das folhas e sua composição. Como o algoritmo escolhe esses valores a partir dos dados sintéticos, eles não são valores de corte clínicos. Aumentar max_depth amplia a flexibilidade, mas também pode favorecer sobreajuste.

5. Treinamento, teste e interpretação da matriz de confusão

O modelo usa a amostra de treino para ajustar parâmetros e regras. As previsões na amostra de teste são comparadas a eventos que o algoritmo não utilizou no ajuste. Esse procedimento é uma avaliação inicial, não uma validação externa nem uma garantia de generalização a novos territórios e períodos.

Na classificação binária, define-se “positivo” como evento = 1. Um verdadeiro positivo (VP) é um evento corretamente identificado; falso negativo (FN) é um evento não detectado; falso positivo (FP) é um não evento classificado como evento; verdadeiro negativo (VN) é um não evento corretamente

classificado.

```
Sensibilidade (recall) = VP / (VP + FN)
Precisão (precision) = VP / (VP + FP)
Especificidade = VN / (VN + FP)
Acurácia = (VP + VN) / total
```

Na matriz produzida pelo scikit-learn, com rótulos 0 e 1, as linhas representam valores observados e as colunas previsões: [[VN, FP], [FN, VP]]. Se o modelo nunca prever a classe positiva, precision_score(..., zero_division=0) retorna zero por convenção operacional; matematicamente, a precisão não é definida quando VP + FP = 0.

6. Como conduzir a explicação passo a passo

- Passo 1: apresente NumPy como ferramenta numérica, pandas como tabela e scikit-learn como conjunto de algoritmos e métricas. Mostre que importar biblioteca não treina um modelo.
- Passo 2: peça que os participantes inspecionem as primeiras linhas e a quantidade de eventos. Explique a diferença entre característica (X) e variável-alvo (y).
- Passo 3: pergunte por que não se deve avaliar um modelo apenas nos registros com os quais ele foi treinado. Explique a estratificação das classes.
- Passo 4: compare a matriz de confusão dos modelos. Destaque que alta acurácia pode coexistir com sensibilidade baixa em eventos raros.
- Passo 5: exiba a árvore e altere max_depth. Pergunte se uma árvore mais detalhada necessariamente faz melhores previsões fora da amostra.

7. Limitações e extensão científica

O conjunto é pequeno, sintético e construído com uma regra simplificada. Não representa o processo gerador de mortalidade neonatal. Em uma aplicação empírica, seria necessário definir temporalmente as variáveis disponíveis na previsão, evitar vazamento de informação, avaliar calibração e discriminação, obter intervalos de incerteza, considerar desbalanceamento e validar o modelo em populações e períodos distintos. Para inferência causal, seriam necessários desenho de pesquisa e pressupostos adicionais.

Perguntas para discussão

- Se um modelo identifica todos os não eventos, mas nenhum evento, sua acurácia pode ser elevada? Por quê?
- Uma divisão da árvore em 35 semanas significa que 35 semanas constitui um limite clínico universal?
- Como a prevalência de eventos na população analisada pode alterar a precisão de um classificador?

Referências para aprofundamento

Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

James, G.; Witten, D.; Hastie, T.; Tibshirani, R.; Taylor, J. (2023). An Introduction to Statistical Learning: with Applications in Python. Springer.

Documentação oficial: https://scikit-learn.org/stable/modules/linear_model.html e https://scikit-learn.org/stable/modules/tree.html

Correspondência com a prática: tutorial de 30 minutos publicado em cbeluzo.github.io/abep-2026/tutoriais/pratica-2h/. Material explicativo; os dados dos exemplos são sintéticos.